

INHA UNIVERSITY IN TASHKENT

SOC 2040 SYSTEM PROGRAMMING

Exceptional Control Flow
Signals (Chapter 8 – Section 8.5, 8.6)

INSTRUCTOR: Jalolliddin Yusupov
PROFESSOR OF COMPUTER SCIENCE & ENG

Problem: Foreground job vs Background job

Foreground job: the shell calls `waitpid` and blocks until the child terminates, then moves to the next command.

Background job (trailing `&`): the shell skips `waitpid` and immediately loops back to read the next command line.

✓ **What works:** The shell correctly waits for and reaps foreground jobs via `waitpid`.

⚠ **Critical flaw:** Background children are *never reaped* — the shell never calls `waitpid` on them.

```
yujboss     2235  0.0  0.0   3216  2004 pts/0    S    21:29   0:00 sleep 500
yujboss     2248  0.0  0.0   7488  3244 pts/0    R+   21:29   0:00 ps aux
yujboss@PositiveHPDesktop:~$ jobs
[1]+  Running                 sleep 500 &
yujboss@PositiveHPDesktop:~$
```

Solution: Exceptional Control Flow (ECF)

"your child died."

① When a background process finishes, the **kernel interrupts** the shell's regular execution to deliver a notification — this is *exceptional control flow*.

② In Unix/Linux, this alert mechanism is called a **signal**. Specifically, the kernel sends `SIGCHLD` to the parent whenever a child terminates or stops.

But: The default action for `SIGCHLD` is to **ignore** the signal.

SIGINT is a signal generated when a user presses Control-C.
This will terminate the program from the terminal.



A signal is a kind of (usually software) interrupt, used to announce **asynchronous** events to a process.

There is a limited list of possible signals; we do not invent our own. (There might be 64 signals, for instance.)
The name of a LINUX signal begins with "SIG".

> *To view all the available signals you can run* ***kill -l*** *to see them. kill command is used to* **execute signals** *from the terminal and also list them.*

Installing Signal Handlers & Concurrency

HOW HANDLERS ARE INVOKED

1. When a process catches signal *k*, the handler installed for *k* is called with a single `int` argument set to *k*.
2. This lets **one function handle multiple signal types** — branch on the argument to distinguish them.
3. **SIGINT example:** the shell sends `SIGINT` when the user types `Ctrl-C`. Default action: terminate process. A custom handler can catch it, print a message, then exit cleanly.

EXAMPLE — SIGINT HANDLER

sigint.c

```c
void handler(int sig) {
  printf("Caught SIGINT!\n");
  exit(0);
}

int main() {
  signal(SIGINT, handler);
  pause(); /* wait for signal */
  exit(0);
}
```

KEY TERMINOLOGY

- **I** **Installing** the handler — passing a function address to `signal()` to change the default action.
- **C** **Catching** the signal — the invocation of the handler when the signal arrives.
- **H** **Handling** the signal — the execution of the handler body.
- **R** **Return** — when the handler executes its `return` statement, control passes back to the interrupted instruction in main. (On some systems, interrupted system calls return with an error instead.)

```
yujboss@PositiveHPDesktop:~$ kill -2 12274
Caught SIGINT!
```

(TRY THIS OUT) Installing Signal Handlers

```c
// multi_signal.c
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>
void handler(int sig) {
    switch(sig) {
        case SIGINT:
            printf("\nCaught SIGINT (Ctrl+C)\n");
            break;
        case SIGTERM:
            printf("\nCaught SIGTERM (termination request)\n");
            break;
        case SIGUSR1:
            printf("\nCaught SIGUSR1 (user-defined 1)\n");
            break;
    }
}
int main() {
    printf("Process %d waiting for signals...\n", getpid());
    printf("Try: Ctrl+C (SIGINT) or kill -TERM %d (SIGTERM)\n\n", getpid());
    signal(SIGINT, handler);
    signal(SIGTERM, handler);
    signal(SIGUSR1, handler);
    while(1) {
        pause();  // Wait for signals
    }
    return 0;
}
```

How to send a signal (Version 1)

ALARM.C — HANDLER

handler

```c
void handler(int sig)
{
  static int beeps = 0;
  printf("BEEP\n");
  if (++beeps < 5)
    alarm(1); /* reschedule */
  else {
    printf("BOOM!\n");
    exit(0);
  }
}
```

📌 The handler is invoked **asynchronously** by each `SIGALRM`, interrupting the infinite `while(1)` in main. When the handler returns, control passes back to main at the exact instruction where it was interrupted.

ALARM.C — MAIN

main

```c
int main()
{
  signal(SIGALRM, handler);
  alarm(1);
  while (1) { ; }
  exit(0);
}
```

OUTPUT

```
$ ./alarm
BEEP
BEEP
BEEP
BEEP
BEEP
BOOM!
```

02 / 04

The signal() Function

FUNCTION SIGNATURE

```
#include <signal.h>
typedef void (*sighandler_t)(int);
    sighandler_t     signal(int signum, sighandler_t handler);
                Returns: ptr to previous handler if OK, SIG_ERR on error (does not set errno)
```

THREE WAYS TO CHANGE A SIGNAL'S ACTION

HANDLER VALUE	EFFECT	
SIG_IGN	Signals of type *signum* are **ignored**.	`// Tell the OS to IGNORE SIGINT (Ctrl+C)` `signal(SIGINT, SIG_IGN);`
SIG_DFL	Action **reverts to the default** for *signum*.	`// Now that we handled it once, let's restore the DEFAULT behavior.` `// The next time the user presses Ctrl+C, the program will just die.`
&my_handler	Address of a **user-defined signal handler** — called whenever the process receives *signum*.	

Signal Handling Examples

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <errno.h>
#include <string.h>
// Error handling function
void unix_error(char *msg) {
    fprintf(stderr, "%s: %s\n", msg,
strerror(errno));
    exit(1);
}
// Handler for SIGINT (Ctrl+C)
void handler1(int sig) {
    if (sig == SIGINT)
        printf("Caught SIGINT (Ctrl+C)\n");
}
// Handler for SIGQUIT (Ctrl+\)
void handler2(int sig) {
    if (sig == SIGQUIT)
        printf("Caught SIGQUIT (Ctrl+\\)\n");
}
```

```c
int main() {
    // Install the SIGINT handler
    printf("Installing SIGINT handler...\n");
    if (signal(SIGINT, handler1) == SIG_ERR) {
        unix_error("signal error for SIGINT");
    }
    // Install the SIGQUIT handler
    printf("Installing SIGQUIT handler...\n");
    if (signal(SIGQUIT, handler2) == SIG_ERR) {
        unix_error("signal error for SIGQUIT");
    }
    printf("MAIN PROCESS NOW WAITING FOR signals...\n");
    printf(" Ctrl+C (SIGINT) or Ctrl+\\ (SIGQUIT)\n");
    printf("Ctrl+Z to suspend or 'kill -9 PID' \n\n");
    // Wait for signals indefinitely
    while (1) {
        pause();  // Suspends until a signal arrives
    }
    exit(0);
}
```

**SIGQUIT → Ctrl **
SIGINT → Ctrl c
SIGSTP → Ctrl z

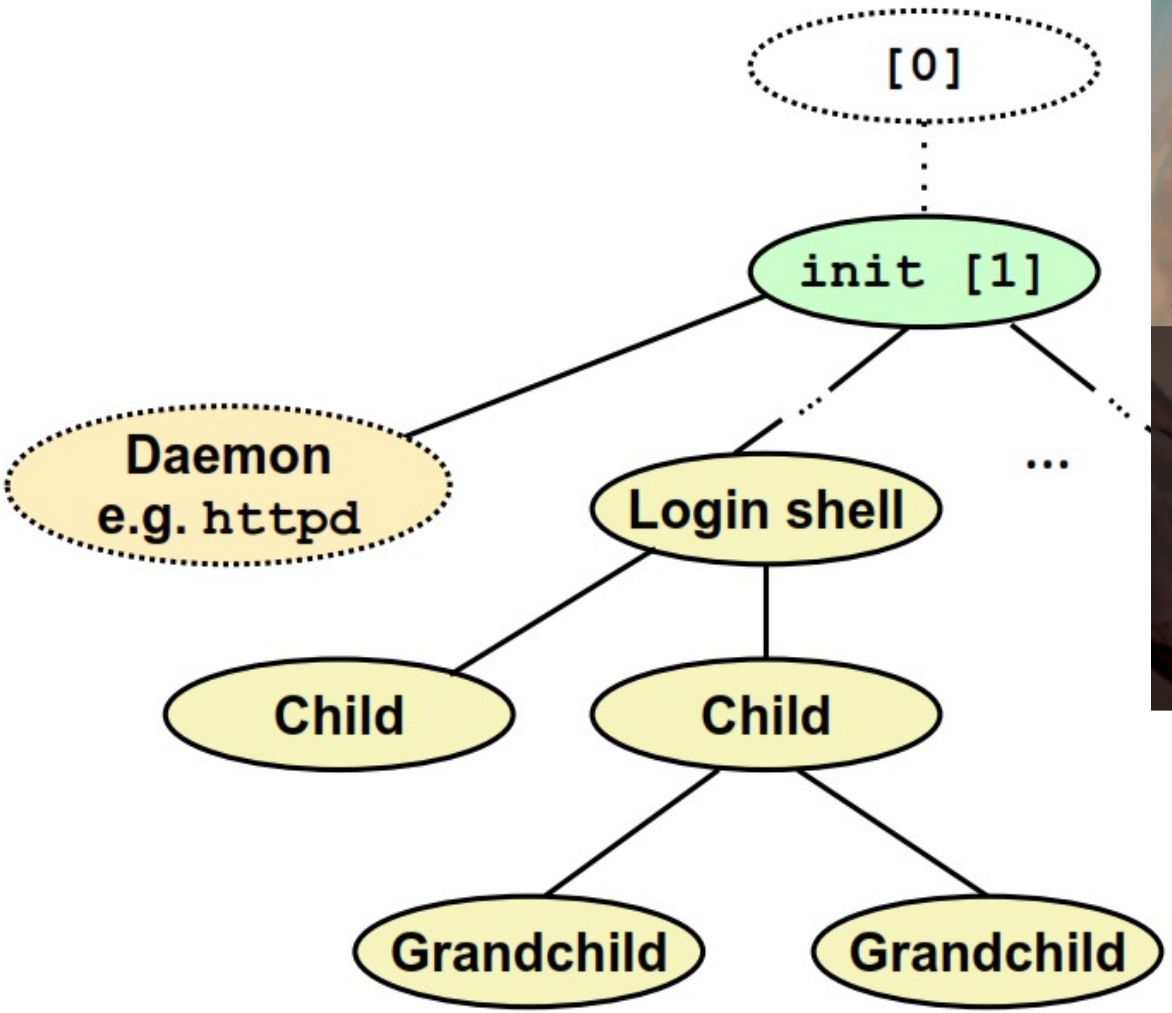
bash

```
1  gcc -o sig2handle sig2handle.c
2  ./sig2handle
```



Questions?

Linux Process Hierarc



Every time you send SIGKILL to a process, you cause this to happen.



Be nice. Send SIGTERM instead.

Note: you can view the hierarchy using the Linux `pstree` command